

以下是为你整理的 **三份标准化交付文档**，已严格对齐当前项目状态、PM 接口规范与 R1 鉴权策略。可直接保存至 docs/ 目录交付。

1. docs/API_CONTRACT.md

```
# 📖 Library Management System - Admin API Contract v1.0

> 📅 版本: `R1-Admin` | 模块: `ADM-01 / ADM-02` | 🌐 Base URL:
`http://localhost:3001`

## 🔑 鉴权说明 (R1 阶段)
- **当前策略**: 所有 `/api/admin/*` 接口 **开放访问, 无需 Token** (便于 R1 功能闭环与
  联动)
- **中间件占位**: 路由层已预留 `checkAdminAuth`, 当前仅执行 `next()` 放行
- **R2 升级路径**: STU/LIB 组输出登录接口后, 将统一接入 `Authorization: Bearer
  <token>` 校验逻辑, 拦截非管理员请求

## 📦 统一响应格式
所有接口均遵循以下 JSON 结构:
```json
{
 "code": 200, // 业务状态码 (200=成功, 201=创建, 400=参数错误, 404=未找到,
 409=冲突, 500=服务器错误)
 "data": { ... }, // 业务数据 (成功时) / null (失败时)
 "msg": "Success" // 人类可读提示
}
```

## 📄 通用查询参数

参数	类型	默认值	说明
page	int	1	页码 (从 1 开始)
size	int	10	每页条数

## 👤 ADM-01: 用户管理 (/api/admin/users)

### 1. 获取用户列表

- **Method:** GET
- **Path:** /api/admin/users
- **Query:** ?page=1&size=10&role=STUDENT&status=ACTIVE
- **Response:**

```
{
 "code": 200,
```

```
"data": {
 "list": [
 {
 "id": "cm...",
 "name": "Alice Zhang",
 "email": "student1@university.edu",
 "studentId": "STU2023001",
 "role": "STUDENT",
 "status": "ACTIVE",
 "createdAt": "2026-04-07T13:58:26.701Z"
 }
],
 "total": 5
},
"msg": "Users retrieved"
}
```

## 2. 创建用户

- **Method:** POST
- **Path:** /api/admin/users
- **Body:**

```
{
 "name": "New User",
 "email": "new@lib.com",
 "studentId": "NEW001",
 "role": "STUDENT",
 "password": "secure123"
}
```

- **Success:** 201 { code: 201, { id: "...", status: "ACTIVE" }, msg: "User created successfully" }
- **Errors:**
  - 400 → {"code":400, "msg":"Missing required fields"} 或 "Invalid email format"
  - 409 → {"code":409, "msg":"studentId already exists"} 或 "email already exists"

## 3. 更新用户

- **Method:** PUT
- **Path:** /api/admin/users/:id
- **Body:** { "name": "...", "email": "...", "role": "...", "status": "..." } (支持部分更新)
- **Success:** 200 { code: 200, { updated: true }, msg: "User updated" }

## 4. 停用/激活用户 (软停用)

- **Method:** DELETE

- **Path:** `/api/admin/users/:id`
  - **Behavior:** 将 `status` 设为 `DEACTIVATED`, 不物理删除记录
  - **Success:** `200 { code: 200, { deleted: true }, msg: "User deactivated" }`
- 

## ADM-02: 图书管理 (`/api/admin/books`)

### 1. 获取图书列表 / 搜索

- **Method:** `GET`
- **Path:** `/api/admin/books`
- **Query:** `?keyword=Clean&genre=Technology&page=1&size=10`
- **Note:** 默认过滤 `isDeleted: false` 的图书
- **Response:**

```
{
 "code": 200,
 "data": {
 "list": [
 {
 "id": "cm...",
 "title": "Clean Code",
 "author": "Robert C. Martin",
 "isbn": "9780132350884",
 "genre": "Technology",
 "stock": 5,
 "isDeleted": false,
 "createdAt": "2026-04-07T13:58:26.701Z"
 }
],
 "total": 20
 },
 "msg": "Books retrieved"
}
```

### 2. 创建图书

- **Method:** `POST`
- **Path:** `/api/admin/books`
- **Body:**

```
{
 "title": "Design Patterns",
 "author": "Gang of Four",
 "isbn": "9780201633610",
 "genre": "Technology",
 "stock": 3,
 "description": "Classic OOP patterns",
 "language": "English",
}
```

```
"shelfLocation": "Tech-A2",
"category": "Architecture"
}
```

- **Success:** 201 { code: 201, { id: "...", stock: 3 }, msg: "Book created" }
- **Errors:**
  - 400 → ISBN 格式错误 / 库存为负数
  - 409 → {"code":409, "msg":"isbn already exists"}

3. 更新图书

- **Method:** PUT
- **Path:** /api/admin/books/:id
- **Body:** { "title": "...", "stock": 10, "category": "..." } (支持部分更新)
- **Success:** 200 { code: 200, { updated: true, stock: 10 }, msg: "Book updated" }

4. 移除图书 (软删除)

- **Method:** DELETE
- **Path:** /api/admin/books/:id
- **Behavior:** 将 isDeleted 设为 true, 前端列表自动隐藏, 关联借阅记录保留
- **Success:** 200 { code: 200, { deleted: true }, msg: "Book removed (soft delete)" }

 枚举与状态码对照

枚举类型	可选值
role	STUDENT   LIBRARIAN   ADMIN
status (User)	ACTIVE   DEACTIVATED
isDeleted (Book)	true   false

💡 下游组可直接基于此契约编写 Mock 数据或自动化测试用例。R2 鉴权接入后, 仅需在请求头追加 `Authorization: Bearer <token>`, 其余契约不变。